ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ИМЕНИ ПАТРИСА ЛУМУМБЫ

Факультет искусственного интеллекта

Отчет

о выполнении лабораторной работы № 6

“Глубокие рекуррентные нейронные сети”

по дисциплине "Нейронные сети"

Вариант задания № 13

Выполнил студент группы ЗФИбд-01-24

Князев А. М._____

Проверил и принял

доцент кафедры прикладного ИИ Леонов С.С._____

с оценкой _____

Москва, 2026

Цель работы: изучение глубоких рекуррентных нейронных сетей для анализа текстовых последовательностей и прогноза временных рядов, а также сопоставление их с одномерными свёрточными сетями.

Основные этапы работы:
1. Построить рекуррентную нейронную сеть (двунаправленную LSTM со слоем векторного представления слов) для распознавания эмоциональной окраски отзывов IMDB. Обучить сеть на последовательностях слов и сопоставить её качество с одномерной свёрточной сетью на тех же данных.
2. Построить рекуррентную сеть для прогноза температуры по набору метеоданных Jena Climate, сформировав обучающие последовательности скользящим окном по временному ряду. Обучить многослойную LSTM-сеть, оценить среднюю абсолютную погрешность прогноза и сравнить её со свёрточно-рекуррентной сетью (комбинацией слоёв Conv1D и LSTM).

Программное обеспечение: Python 3.x, Jupyter Notebook, библиотеки PyTorch, pandas, NumPy и Matplotlib.

Сценарий выполнения работы

Этап 1. Двунаправленная LSTM для классификации отзывов IMDB

Набор данных IMDB содержит текстовые рецензии на фильмы с двумя метками тональности. Тексты приводятся к нижнему регистру и токенизируются, строится словарь из 12000 наиболее частых слов; каждый отзыв превращается в последовательность индексов слов. Всего использовано 3600 текстов. Последовательности дополняются (padding) до общей длины и подаются в слой векторного представления (Embedding). Данные разделены на обучающее, контрольное и тестовое подмножества.

Процесс обучения (общий для этапов работы). Сеть обучается мини-батчами методом обратного распространения ошибки во времени (BPTT) с оптимизатором Adam. На каждой эпохе по обучающему подмножеству вычисляются градиенты функции потерь и обновляются веса, а по контрольному подмножеству контролируется качество. Для бинарной классификации используется функция потерь BCEWithLogits и метрика

accuracy; для прогноза температуры (этап 2) — MSE и MAE.

Согласно варианту 13 сеть содержит слой Embedding размерности 64 и два рекуррентных слоя LSTM по 32 нейрона в двунаправленном режиме; выход двух направлений объединяется и подаётся на линейный выходной нейрон. Функция потерь — бинарная перекрёстная энтропия (BCEWithLogits), оптимизатор — Adam, число эпох — 30.

Таблица 1 — Структура сети (этап 1, вариант 13)

Параметр	**Значение**
Размер словаря	12000
Размерность Embedding	64
Рекуррентные слои	2 × LSTM по 32 нейрона, двунаправленные
Прореживание	Dropout 0.2
Выходной слой	1 нейрон (логит)
Оптимизатор / потери / эпохи	Adam / BCEWithLogits / 30

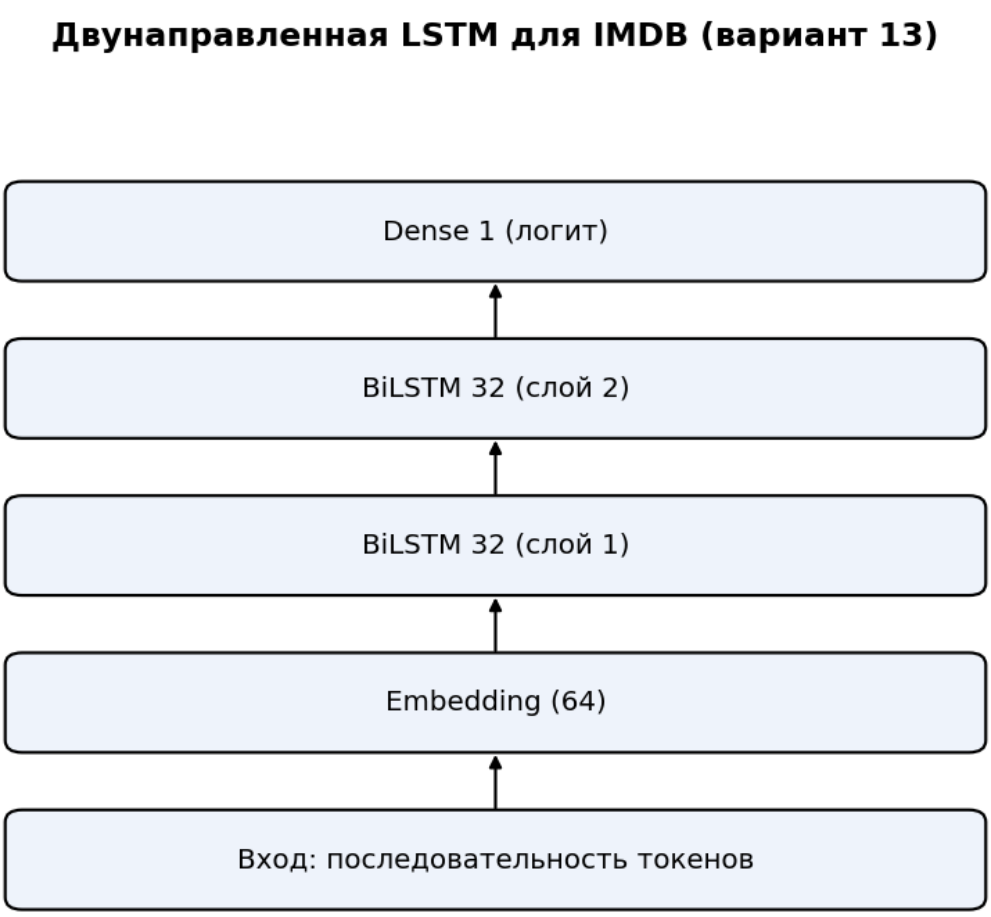


Рис. 1 — Схема двунаправленной LSTM для IMDB (вариант 13)



Рис. 2 — Схема одномерной свёрточной сети для IMDB

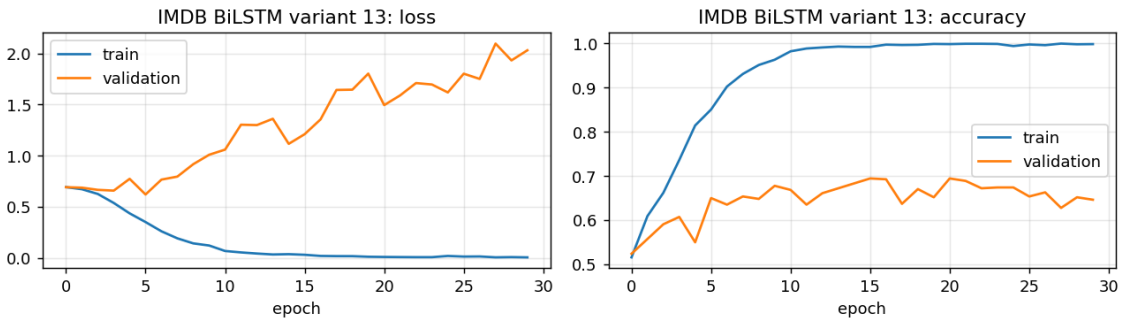


Рис. 3 — Графики функции потерь и точности (IMDB, двунаправленная LSTM)

Для сопоставления построена одномерная свёрточная сеть с тремя свёрточными слоями (согласно пункту задания), обученная на тех же данных.

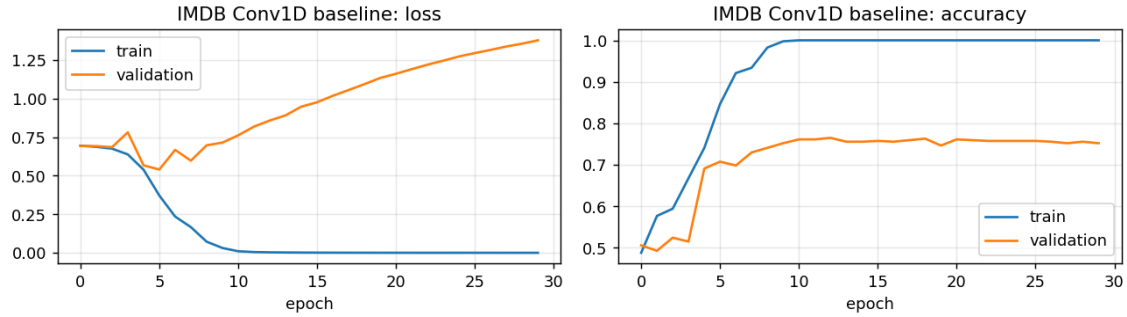


Рис. 4 — Графики функции потерь и точности (IMDB, свёрточная сеть Conv1D)

Таблица 2 — Сопоставление моделей (этап 1)

Модель	**Test loss**	**Test accuracy**
Двунаправленная LSTM (вариант 13)	2.0349	0.6481
Одномерная свёрточная сеть (Conv1D)	1.4448	0.7426

Обе модели обучаются на небольшой выборке (3600 текстов) и склонны к переобучению. В данном прогоне свёрточная сеть показала более высокую точность на тесте (0.74), чем двунаправленная LSTM (0.65). Точность ограничена объёмом данных и заданными вариантом параметрами сети.

Этап 2. Прогноз температуры по набору Jena Climate

Набор данных Jena Climate содержит метеорологические измерения (420451 запись, 14 признаков). Данные нормализуются; формируются окна наблюдений за 10 дней (по одному значению в час, 240 шагов) с задержкой прогноза в один день. Выборки разделены на обучающее, контрольное и тестовое подмножества (2500 / 1250 / 1250 окон).

Согласно варианту 13 сеть содержит три рекуррентных слоя LSTM по 64 нейрона в двунаправленном режиме с прореживанием 0.3; выход подаётся на линейный нейрон, прогнозирующий температуру. Функция потерь — среднеквадратическая погрешность, метрика — средняя абсолютная погрешность (MAE, °C), оптимизатор — Adam, число эпох — 30.

Таблица 3 — Структура сети (этап 2, вариант 13)

Параметр	**Значение**
Размер окна / шаг	240 шагов (10 дней, 1 значение в час)
Число признаков	14
Рекуррентные слои	3 × LSTM по 64 нейрона, двунаправленные
Прореживание	Dropout 0.3
Выходной слой	1 нейрон (температура)
Оптимизатор / потери / метрика	Adam / MSE / MAE

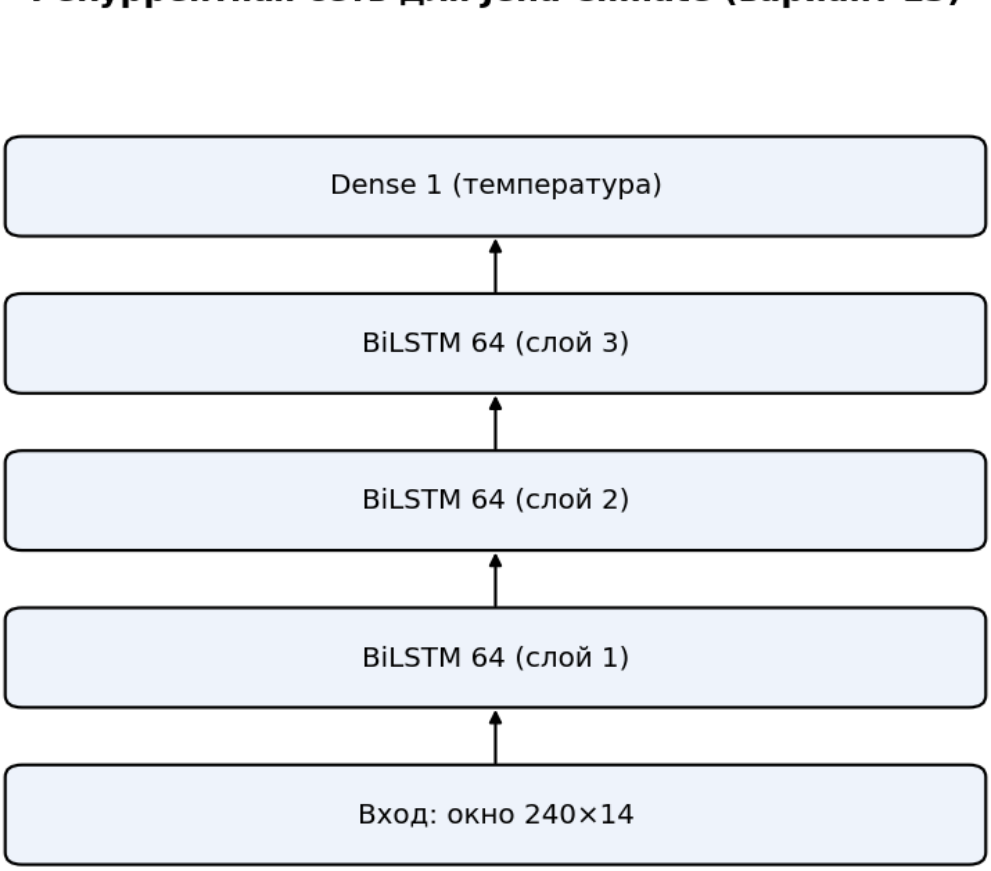


Рис. 5 — Схема рекуррентной сети для Jena Climate (вариант 13)



Рис. 6 — Схема свёрточно-рекуррентной сети для Jena Climate

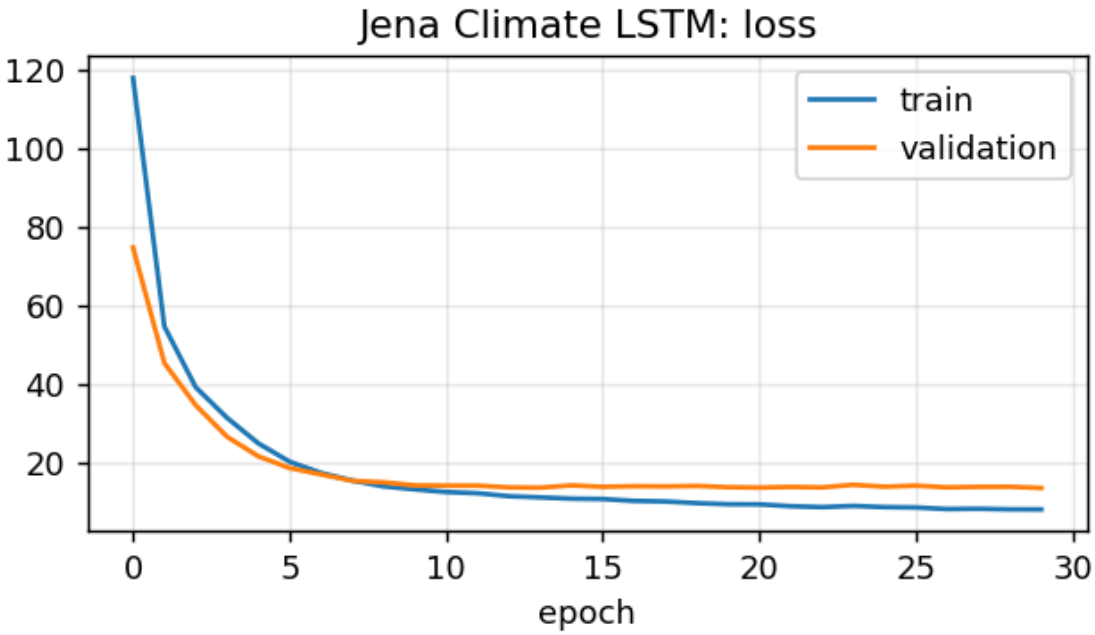


Рис. 7 — График функции потерь (Jena Climate, двунаправленная LSTM)

Для сопоставления построена сеть, первый блок которой — одномерная свёрточная сеть из трёх свёрточных слоёв, а второй блок — рекуррентный (LSTM) согласно варианту.

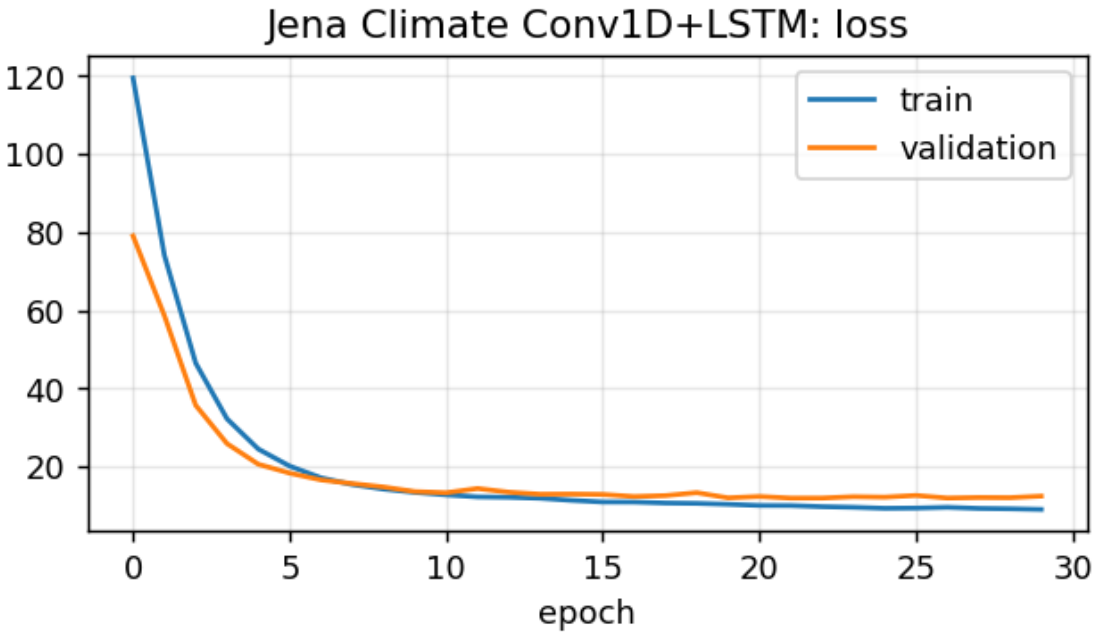


Рис. 8 — График функции потерь (Jena Climate, Conv1D + LSTM)

Таблица 4 — Сопоставление моделей (этап 2)

Модель	**Test loss (MSE)**	**Test MAE, °C**
Двунаправленная LSTM (вариант 13)	13.0723	2.80
Conv1D + LSTM	10.6461	2.54

Обе сети прогнозируют температуру со средней абсолютной погрешностью около 2.5–2.8 °C. Свёрточно-рекуррентная модель в данном прогоне показала немного меньшую погрешность (2.54 °C против 2.80 °C): свёрточный блок выделяет локальные признаки последовательности перед рекуррентным слоем.

Код программы

```
import random
import re
import zipfile
from collections import Counter
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.nn.utils.rnn import pad_sequence
from torch.utils.data import DataLoader, Dataset, TensorDataset,
random_split

DEVICE = torch.device(
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
SEED = 13
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

TOKEN_RE = re.compile(r"[a-zA-Z']+")
print(f"Device: {DEVICE}")

def plot_history(history: dict[str, list[float]], title: str,
show_accuracy: bool = True) -> None:
    cols = 2 if show_accuracy else 1
    fig, axes = plt.subplots(1, cols, figsize=(10 if show_accuracy else 5,
3))
    if not show_accuracy:
        axes = [axes]

    axes[0].plot(history["loss"], label="train")
    axes[0].plot(history["val_loss"], label="validation")
    axes[0].set_title(f"{title}: loss")
    axes[0].set_xlabel("epoch")
    axes[0].grid(True, alpha=0.3)
    axes[0].legend()

    if show_accuracy:
        axes[1].plot(history["acc"], label="train")
        axes[1].plot(history["val_acc"], label="validation")
        axes[1].set_title(f"{title}: accuracy")
        axes[1].set_xlabel("epoch")
```

```
        axes[1].grid(True, alpha=0.3)
        axes[1].legend()

    plt.tight_layout()
    plt.show()

def tokenize(text: str) -> list[str]:
    return TOKEN_RE.findall(text.lower())


def read_imdb(zip_path: Path, max_docs_per_class: int = 900) ->
tuple[list[str], list[int]]:
    if not zip_path.exists():
        raise FileNotFoundError(f"Не найден архив IMDB: {zip_path}")

    texts = []
    labels = []
    with zipfile.ZipFile(zip_path) as archive:
        for split in ["train", "test"]:
            for label_name, label in [("neg", 0), ("pos", 1)]:
                prefix = f"aclImdb/{split}/{label_name}/"
                names = [n for n in archive.namelist() if
n.startswith(prefix) and n.endswith(".txt")]
                names = [n for n in names if "/._" not in n]
                for name in names[:max_docs_per_class]:
                    texts.append(archive.read(name).decode("utf-8",
errors="ignore"))
                    labels.append(label)
    return texts, labels


def make_vocab(texts: list[str], max_words: int = 12000) -> dict[str,
int]:
    counter = Counter()
    for text in texts:
        counter.update(tokenize(text))
    vocab = {"<pad>": 0, "<unk>": 1}
    for word, _ in counter.most_common(max_words - 2):
        vocab[word] = len(vocab)
    return vocab

class TextDataset(Dataset):
    def __init__(self, texts: list[str], labels: list[int], vocab:
dict[str, int], max_len: int = 180):
        self.labels = labels
        self.samples = []
        for text in texts:
            ids = [vocab.get(token, 1) for token in tokenize(text)
[:max_len]]
            self.samples.append(torch.tensor(ids or [1],
```

```
dtype=torch.long))

    def __len__(self) -> int:
        return len(self.samples)

    def __getitem__(self, index: int) -> tuple[torch.Tensor,
torch.Tensor]:
        return self.samples[index],
torch.tensor(float(self.labels[index]), dtype=torch.float32)


def collate_batch(batch: list[tuple[torch.Tensor, torch.Tensor]]) ->
tuple[torch.Tensor, torch.Tensor]:
    texts, labels = zip(*batch)
    return pad_sequence(texts, batch_first=True, padding_value=0),
torch.stack(labels)

class BiLstmClassifier(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int = 64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.rnn = nn.LSTM(
            input_size=embed_dim,
            hidden_size=32,
            num_layers=2,
            batch_first=True,
            bidirectional=True,
            dropout=0.2,
        )
        self.output = nn.Linear(64, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        embedded = self.embedding(x)
        _, (hidden, _) = self.rnn(embedded)
        last_forward = hidden[-2]
        last_backward = hidden[-1]
        return self.output(torch.cat([last_forward, last_backward],
dim=1)).squeeze(1)


class ConvTextClassifier(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int = 64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.features = nn.Sequential(
            nn.Conv1d(embed_dim, 64, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.MaxPool1d(2),
```

```
            nn.Conv1d(64, 64, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Conv1d(64, 64, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.AdaptiveMaxPool1d(1),
        )
        self.output = nn.Linear(64, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        embedded = self.embedding(x).transpose(1, 2)
        features = self.features(embedded).squeeze(-1)
        return self.output(features).squeeze(1)

def evaluate_binary(model: nn.Module, loader: DataLoader, criterion:
nn.Module) -> tuple[float, float]:
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total_loss += float(criterion(logits, y).item()) * len(x)
            correct += int(((torch.sigmoid(logits) >= 0.5).float() ==
y).sum().item())
            total += len(x)
    return total_loss / total, correct / total


def train_binary(model: nn.Module, train_loader: DataLoader, val_loader:
DataLoader, epochs: int) -> dict[str, list[float]]:
    model.to(DEVICE)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    history = {"loss": [], "val_loss": [], "acc": [], "val_acc": []}

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        correct = 0
        total = 0
        for x, y in train_loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            optimizer.zero_grad()
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward()
            optimizer.step()
            total_loss += float(loss.item()) * len(x)
```

```
            correct += int(((torch.sigmoid(logits) >= 0.5).float() ==
y).sum().item())
            total += len(x)

        val_loss, val_acc = evaluate_binary(model, val_loader, criterion)
        history["loss"].append(total_loss / total)
        history["acc"].append(correct / total)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
        print(f"epoch {epoch + 1:02d}: loss={history['loss'][-1]:.4f},
acc={history['acc'][-1]:.4f}, val_acc={val_acc:.4f}")

    return history

texts, labels = read_imdb(Path("../IMDB/aclImdb.zip"),
max_docs_per_class=900)
vocab = make_vocab(texts, max_words=12000)
text_dataset = TextDataset(texts, labels, vocab, max_len=180)

train_size = int(0.7 * len(text_dataset))
val_size = int(0.15 * len(text_dataset))
test_size = len(text_dataset) - train_size - val_size
text_train, text_val, text_test = random_split(
    text_dataset, [train_size, val_size, test_size],
generator=torch.Generator().manual_seed(SEED)
)

text_train_loader = DataLoader(text_train, batch_size=64, shuffle=True,
collate_fn=collate_batch)
text_val_loader = DataLoader(text_val, batch_size=128, shuffle=False,
collate_fn=collate_batch)
text_test_loader = DataLoader(text_test, batch_size=128, shuffle=False,
collate_fn=collate_batch)

print(f"Texts: {len(text_dataset)}, vocab: {len(vocab)}")

bilstm_model = BiLstmClassifier(len(vocab), embed_dim=64)
BILSTM_EPOCHS = 30
bilstm_history = train_binary(bilstm_model, text_train_loader,
text_val_loader, epochs=BILSTM_EPOCHS)

bilstm_loss, bilstm_acc = evaluate_binary(bilstm_model, text_test_loader,
nn.BCEWithLogitsLoss())
print(f"BiLSTM variant: test_loss={bilstm_loss:.4f},
test_acc={bilstm_acc:.4f}")
plot_history(bilstm_history, "IMDB BiLSTM variant 13")

conv_text_model = ConvTextClassifier(len(vocab), embed_dim=64)
CONV_TEXT_EPOCHS = 30
conv_text_history = train_binary(conv_text_model, text_train_loader,
```

```
text_val_loader, epochs=CONV_TEXT_EPOCHS)

conv_loss, conv_acc = evaluate_binary(conv_text_model, text_test_loader,
nn.BCEWithLogitsLoss())
print(f"Conv1D baseline: test_loss={conv_loss:.4f},
test_acc={conv_acc:.4f}")
plot_history(conv_text_history, "IMDB Conv1D baseline")

def read_jena(zip_path: Path) -> pd.DataFrame:
    if not zip_path.exists():
        raise FileNotFoundError(f"Не найден архив Jena Climate:
{zip_path}")
    with zipfile.ZipFile(zip_path) as archive:
        csv_name = next(name for name in archive.namelist() if
name.endswith("jena_climate_2009_2016.csv"))
        with archive.open(csv_name) as file:
            return pd.read_csv(file)


jena = read_jena(Path("../Jena Climate/jena_climate.zip"))
print(jena.shape)
jena.head()

plt.figure(figsize=(12, 3))
plt.plot(jena["T (degC)"].values[:2000])
plt.title("Jena Climate: температура, первые 2000 измерений")
plt.xlabel("Номер измерения")
plt.ylabel("T (degC)")
plt.grid(True, alpha=0.3)
plt.show()

feature_frame = jena.drop(columns=["Date Time"])
feature_names = list(feature_frame.columns)
target_index = feature_names.index("T (degC)")
raw_values = feature_frame.values.astype("float32")

train_border = int(0.7 * len(raw_values))
mean = raw_values[:train_border].mean(axis=0)
std = raw_values[:train_border].std(axis=0)
values = (raw_values - mean) / std
target = raw_values[:, target_index]

LOOKBACK = 1440      # 10 дней назад при шаге исходных данных 10 минут
DELAY = 144          # прогноз на 1 день вперед
STEP = 6             # одно значение в час
BATCH_SIZE = 128
MAX_SAMPLES = 2500


def make_climate_tensors(start: int, end: int, max_samples: int) ->
```

```
tuple[torch.Tensor, torch.Tensor]:
    last_start = min(end - DELAY, len(values) - DELAY)
    indices = np.arange(start + LOOKBACK, last_start)
    if len(indices) > max_samples:
        rng = np.random.default_rng(SEED)
        indices = np.sort(rng.choice(indices, size=max_samples,
replace=False))

    samples = np.stack([values[i - LOOKBACK:i:STEP] for i in
indices]).astype("float32")
    labels = target[indices + DELAY].astype("float32").reshape(-1, 1)
    return torch.tensor(samples), torch.tensor(labels)


x_train, y_train = make_climate_tensors(0, train_border, MAX_SAMPLES)
x_val, y_val = make_climate_tensors(train_border, int(0.85 * len(values)),
MAX_SAMPLES // 2)
x_test, y_test = make_climate_tensors(int(0.85 * len(values)),
len(values), MAX_SAMPLES // 2)

climate_train_loader = DataLoader(TensorDataset(x_train, y_train),
batch_size=BATCH_SIZE, shuffle=True)
climate_val_loader = DataLoader(TensorDataset(x_val, y_val),
batch_size=BATCH_SIZE, shuffle=False)
climate_test_loader = DataLoader(TensorDataset(x_test, y_test),
batch_size=BATCH_SIZE, shuffle=False)

print(x_train.shape, x_val.shape, x_test.shape)

class ClimateLstm(nn.Module):
    def __init__(self, input_size: int):
        super().__init__()
        self.rnn = nn.LSTM(input_size, 64, num_layers=3, batch_first=True,
bidirectional=True, dropout=0.3)
        self.output = nn.Linear(128, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        _, (hidden, _) = self.rnn(x)
        return self.output(torch.cat([hidden[-2], hidden[-1]], dim=1))


class ClimateConvLstm(nn.Module):
    def __init__(self, input_size: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(input_size, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv1d(32, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv1d(32, 32, kernel_size=3, padding=1),
```

```
            nn.ReLU(),
        )
        self.rnn = nn.LSTM(32, 64, num_layers=3, batch_first=True,
bidirectional=True, dropout=0.3)
        self.output = nn.Linear(128, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        features = self.conv(x.transpose(1, 2)).transpose(1, 2)
        _, (hidden, _) = self.rnn(features)
        return self.output(torch.cat([hidden[-2], hidden[-1]], dim=1))

def evaluate_regression(model: nn.Module, loader: DataLoader, criterion:
nn.Module) -> tuple[float, float]:
    model.eval()
    total_loss = 0.0
    total_abs = 0.0
    total = 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            pred = model(x)
            total_loss += float(criterion(pred, y).item()) * len(x)
            total_abs += float(torch.abs(pred - y).sum().item())
            total += len(x)
    return total_loss / total, total_abs / total


def train_regression(model: nn.Module, train_loader: DataLoader,
val_loader: DataLoader, epochs: int) -> dict[str, list[float]]:
    model.to(DEVICE)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    history = {"loss": [], "val_loss": []}

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        total = 0
        for x, y in train_loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            optimizer.zero_grad()
            pred = model(x)
            loss = criterion(pred, y)
            loss.backward()
            optimizer.step()
            total_loss += float(loss.item()) * len(x)
            total += len(x)

        val_loss, val_mae = evaluate_regression(model, val_loader,
criterion)
        history["loss"].append(total_loss / total)
```

```
        history["val_loss"].append(val_loss)
        print(f"epoch {epoch + 1:02d}: loss={history['loss'][-1]:.4f},
val_loss={val_loss:.4f}, val_mae={val_mae:.2f}")

    return history

climate_model = ClimateLstm(input_size=x_train.shape[-1])
CLIMATE_EPOCHS = 30
climate_history = train_regression(climate_model, climate_train_loader,
climate_val_loader, epochs=CLIMATE_EPOCHS)

climate_test_loss, climate_test_mae = evaluate_regression(climate_model,
climate_test_loader, nn.MSELoss())
print(f"Jena LSTM: test_loss={climate_test_loss:.4f},
test_mae={climate_test_mae:.2f}")
plot_history(climate_history, "Jena Climate LSTM", show_accuracy=False)

climate_conv_model = ClimateConvLstm(input_size=x_train.shape[-1])
conv_climate_history = train_regression(climate_conv_model,
climate_train_loader, climate_val_loader, epochs=CLIMATE_EPOCHS)

conv_climate_loss, conv_climate_mae =
evaluate_regression(climate_conv_model, climate_test_loader, nn.MSELoss())
print(f"Jena Conv1D+LSTM: test_loss={conv_climate_loss:.4f},
test_mae={conv_climate_mae:.2f}")
plot_history(conv_climate_history, "Jena Climate Conv1D+LSTM",
show_accuracy=False)
```

Вывод

В ходе работы изучены глубокие рекуррентные нейронные сети для двух типов задач — анализа текстовых последовательностей и прогноза временного ряда, а их результаты сопоставлены со свёрточными и свёрточно-рекуррентными сетями.

На этапе 1 двунаправленная сеть LSTM (вариант 13: Embedding 64, два слоя по 32 нейрона) для классификации отзывов IMDB достигла точности 0.6481, а одномерная свёрточная сеть на тех же данных — 0.7426. На небольшой выборке обе модели склонны к переобучению, и результат рекуррентной сети ограничен заданными вариантом параметрами; свёрточная сеть на этой задаче оказалась точнее и обучается быстрее.

На этапе 2 трёхслойная двунаправленная сеть LSTM для прогноза температуры Jena Climate показала среднюю абсолютную погрешность около 2.80 °C, а свёрточно-рекуррентная сеть (Conv1D + LSTM) — 2.54 °C. Комбинированная архитектура оказалась точнее: свёрточные слои выделяют локальные признаки временного ряда, а рекуррентные учитывают их временную динамику.

Таким образом, рекуррентные и свёрточно-рекуррентные сети успешно решают задачи классификации текстов и прогноза временного ряда, показывая результаты, сопоставимые со свёрточными моделями, а выбор архитектуры заметно влияет на точность. Все этапы работы выполнены, получены графики функции потерь и метрик качества, проведено сопоставление со свёрточными сетями.